Crime Count Forecasting per District

IT462

Student Name	IDs	Email
Raghad Almutairi	443200793	443200793@student.ksu.edu.sa
Sarah Alruwayte	444200758	444200758@student.ksu.edu.sa
Norah Alfaheed	444200779	444200779@student.ksu.edu.sa
Atheer Budie	444200894	444200894@student.ksu.edu.sa

Section: 2766
Supervised by: Dr. AFSHAN JAFRI

List of Content

List of Tables

List of Figures

1. Introduction

This phase applies Spark SQL and DataFrame operations [1] to perform structured analytical queries on the Chicago Crimes dataset. SQL queries were designed to answer specific questions about spatial, temporal, and behavioral crime patterns that directly support our regression-based crime forecasting.

All queries are executed on two datasets produced in the preprocessing Phase. The cleaned dataset (crimes_cleaned) contains 952,563 incident-level records and is used for queries requiring individual crime attributes such as arrest status, domestic indicator, and crime type. The preprocessed dataset (crimes_weekly) contains 4,644 aggregated weekly rows and is used for queries involving temporal trends and weekly crime statistics.

2. Dataset Overview

Before executing SQL queries, both datasets were loaded into Spark DataFrames and registered as temporary SQL views [1]. This section documents the structure, size, and sample content of each dataset to provide context for the queries that follow.

Although two tables (crimes_cleaned and crimes_weekly) are used in the analysis, they originate from the same dataset. The crimes_weekly table is a preprocessed and aggregated version of crimes_cleaned produced during the data transformation phase.
Therefore, performing a join between them would not provide additional analytical value and could introduce redundant or duplicated information. For this reason, the queries focus on analytical operations such as aggregations, window functions, statistical summaries, and temporal analysis instead of joins.

2.1 Cleaned Dataset (crimes_cleaned)

The cleaned dataset is the output of the Phase 2 cleaning pipeline. It contains all 952,563 incident-level records after removing rows with missing geographic fields from the original 971,865 records. It retains all 22 original columns and serves as the source for queries that require individual crime attributes.

Table 1: Cleaned dataset key attributes

Attribute	Type	Description
id	Integer	Unique record identifier
date	Timestamp	Incident occurrence date and time
primary_type	String	Main crime category (~30 types)
district	Integer	Police district number (1–31)
arrest	Boolean	Whether an arrest was made
domestic	Boolean	Whether the incident was domestic-related
year	Integer	Year of the incident (2021–2024)

```
scala> // ============================================
     | // PHASE 4 — SQL OPERATIONS IN APACHE SPARK
     | // STEP 1: Load Cleaned Dataset
     | // ============================================
     |
     | val cleanedDF = spark.read
     |   .option("header", "true")
     |   .option("inferSchema", "true")
     |   .csv("/Users/raghadfares/Desktop/BigData/clean_data.csv")
     |
     | println(s"Cleaned dataset rows: ${cleanedDF.count()}")
     | println(s"Cleaned dataset columns: ${cleanedDF.columns.length}")
Cleaned dataset rows: 952563
Cleaned dataset columns: 22
val cleanedDF: org.apache.spark.sql.DataFrame = [id: int, case_number: string ... 20 more fields]
```

Figure 1: Spark shell output showing Step 1 – Cleaned dataset

```
scala> // ============================================
     | // STEP 2: Load Preprocessed Dataset
     | // ============================================
     |
     | val preprocessedDF = spark.read
     |   .option("header", "true")
     |   .option("inferSchema", "true")
     |   .csv("/Users/raghadfares/Desktop/BigData/preprocessed_Data.csv")
     |
     | println(s"Preprocessed dataset rows: ${preprocessedDF.count()}")
     | println(s"Preprocessed dataset columns: ${preprocessedDF.columns.length}")
Preprocessed dataset rows: 4644
Preprocessed dataset columns: 38
val preprocessedDF: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 36 more
fields]
```

Figure 2: Spark shell output showing Step 2 – Preprocessed dataset

```
scala> // ============================================
     | // STEP 3: Register as SQL Views
     | // ============================================
     |
     | cleanedDF.createOrReplaceTempView("crimes_cleaned")
     | preprocessedDF.createOrReplaceTempView("crimes_weekly")
     |
     | println("Both datasets registered as SQL views successfully.")
     | println("crimes_cleaned -> cleaned dataset (952,563 records)")
     | println("crimes_weekly  -> preprocessed weekly dataset (4,644 rows)")
26/03/14 15:40:21 WARN SparkStringUtils: Truncated the string representation of a plan since it was too
 large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
Both datasets registered as SQL views successfully.
crimes_cleaned -> cleaned dataset (952,563 records)
crimes_weekly  -> preprocessed weekly dataset (4,644 rows)
```

Figure 3: Spark shell output showing Step 3 – SQL view registration

2.2 Preprocessed Dataset (crimes_weekly)

The preprocessed dataset is the final output of the Phase 2 aggregation pipeline. It contains 4,644 aggregated weekly rows representing the total crime count per district per week across 2021–2024. It has 38 columns including individual crime-type pivot columns, engineered temporal features, and the lag feature used for forecasting.

Note: The preprocessed CSV file contains 4,644 rows. The 23-row difference from the 4,667 rows reported in the preprocessing phase is attributable to District 31, which has only 57 total crimes across the entire 2021–2024 period. Weeks with zero crime counts for this district were excluded during CSV export, resulting in 46 weekly records for District 31 instead of the full 209. This difference has no impact on the analytical results, as District 31 contributes a negligible share of city-wide crime activity.

Table 2: Preprocessed dataset key attributes

Attribute	Type	Description
district	Integer	Police district number
week_start	Timestamp	Start date of the aggregation week
month	Integer	Month number (1–12)
week_no	Integer	Week number within the year (1–52)
THEFT / BATTERY / ROBBERY ...	Integer	Weekly count per crime type (30 columns)
total_crime_count	Integer	Total crimes across all types for that week
district_indexed	Double	StringIndexer-encoded district category
prev_week_total	Integer	Total crimes from the previous week (lag feature)

```
scala> // ============================================
     | // STEP 4: Print Schemas
     | // ============================================
     |
     | println("--- Cleaned Dataset Schema ---")
     | cleanedDF.printSchema()
     |
     | println("--- Preprocessed Dataset Schema ---")
     | preprocessedDF.printSchema()
--- Cleaned Dataset Schema ---
root
 |-- id: integer (nullable = true)
 |-- case_number: string (nullable = true)
 |-- date: timestamp (nullable = true)
 |-- block: string (nullable = true)
 |-- iucr: string (nullable = true)
 |-- primary_type: string (nullable = true)
 |-- description: string (nullable = true)
 |-- location_description: string (nullable = true)
 |-- arrest: boolean (nullable = true)
 |-- domestic: boolean (nullable = true)
 |-- beat: integer (nullable = true)
 |-- district: integer (nullable = true)
 |-- ward: integer (nullable = true)
 |-- community_area: integer (nullable = true)
 |-- fbi_code: string (nullable = true)
 |-- x_coordinate: integer (nullable = true)
 |-- y_coordinate: integer (nullable = true)
 |-- year: integer (nullable = true)
 |-- updated_on: timestamp (nullable = true)
 |-- latitude: double (nullable = true)
 |-- longitude: double (nullable = true)
 |-- location: string (nullable = true)
```

Figure 4: Spark shell output showing Step 4 – Cleaned dataset schema

```
--- Preprocessed Dataset Schema ---
root
 |-- district: integer (nullable = true)
 |-- week_start: timestamp (nullable = true)
 |-- ARSON: integer (nullable = true)
 |-- ASSAULT: integer (nullable = true)
 |-- BATTERY: integer (nullable = true)
 |-- BURGLARY: integer (nullable = true)
 |-- CONCEALED CARRY LICENSE VIOLATION: integer (nullable = true)
 |-- CRIMINAL DAMAGE: integer (nullable = true)
 |-- CRIMINAL SEXUAL ASSAULT: integer (nullable = true)
 |-- CRIMINAL TRESPASS: integer (nullable = true)
 |-- DECEPTIVE PRACTICE: integer (nullable = true)
 |-- GAMBLING: integer (nullable = true)
 |-- HOMICIDE: integer (nullable = true)
 |-- HUMAN TRAFFICKING: integer (nullable = true)
 |-- INTERFERENCE WITH PUBLIC OFFICER: integer (nullable = true)
 |-- INTIMIDATION: integer (nullable = true)
 |-- KIDNAPPING: integer (nullable = true)
 |-- LIQUOR LAW VIOLATION: integer (nullable = true)
 |-- MOTOR VEHICLE THEFT: integer (nullable = true)
 |-- NARCOTICS: integer (nullable = true)
 |-- NON-CRIMINAL: integer (nullable = true)
 |-- OBSCENITY: integer (nullable = true)
 |-- OFFENSE INVOLVING CHILDREN: integer (nullable = true)
 |-- OTHER NARCOTIC VIOLATION: integer (nullable = true)
 |-- OTHER OFFENSE: integer (nullable = true)
 |-- PROSTITUTION: integer (nullable = true)
 |-- PUBLIC INDECENCY: integer (nullable = true)
 |-- PUBLIC PEACE VIOLATION: integer (nullable = true)
 |-- ROBBERY: integer (nullable = true)
 |-- SEX OFFENSE: integer (nullable = true)
 |-- STALKING: integer (nullable = true)
 |-- THEFT: integer (nullable = true)
 |-- WEAPONS VIOLATION: integer (nullable = true)
 |-- total_crime_count: integer (nullable = true)
 |-- month: integer (nullable = true)
 |-- week_no: integer (nullable = true)
 |-- district_indexed: double (nullable = true)
 |-- prev_week_total: integer (nullable = true)
```

Figure 5: Spark shell output showing Step 4 – Preprocessed dataset schema

```
scala> // ============================================
     | // STEP 5: Print Sample Rows
     | // ============================================
     |
     | println("--- Cleaned Dataset Sample (5 rows, key columns) ---")
     | cleanedDF.select(
     |   "id", "date", "primary_type", "district",
     |   "arrest", "domestic", "year"
     | ).show(5, truncate = false)
     |
     | println("--- Preprocessed Dataset Sample (5 rows, key columns) ---")
     | preprocessedDF.select(
     |   "district", "week_start", "month", "week_no",
     |   "THEFT", "BATTERY", "ROBBERY",
     |   "total_crime_count", "prev_week_total"
     | ).show(5, truncate = false)
--- Cleaned Dataset Sample (5 rows, key columns) ---
+--------+-------------------+------------------+--------+------+--------+----+
|id      |date               |primary_type      |district|arrest|domestic|year|
+--------+-------------------+------------------+--------+------+--------+----+
|12419690|2021-07-07 10:30:00|SEX OFFENSE       |5       |false |false   |2021|
|12342615|2021-04-17 15:20:00|ROBBERY           |6       |false |false   |2021|
|26262   |2021-09-08 16:45:00|HOMICIDE          |11      |true  |false   |2021|
|12298539|2021-02-22 13:36:00|ROBBERY           |10      |false |false   |2021|
|12346572|2021-04-22 14:30:00|DECEPTIVE PRACTICE|7       |true  |false   |2021|
+--------+-------------------+------------------+--------+------+--------+----+
```

Figure 6: Spark shell output showing Step 5 – Cleaned dataset sample rows

```
only showing top 5 rows
--- Preprocessed Dataset Sample (5 rows, key columns) ---
+--------+-------------------+-----+-------+-----+-------+-------+-----------------+---------------+
|district|week_start         |month|week_no|THEFT|BATTERY|ROBBERY|total_crime_count|prev_week_total|
+--------+-------------------+-----+-------+-----+-------+-------+-----------------+---------------+
|1       |2021-01-04 00:00:00|1    |1      |39   |17     |7      |123              |47             |
|1       |2021-01-11 00:00:00|1    |2      |39   |18     |5      |124              |123            |
|1       |2021-01-18 00:00:00|1    |3      |39   |13     |8      |118              |124            |
|1       |2021-01-25 00:00:00|1    |4      |21   |18     |7      |96               |118            |
|1       |2021-02-01 00:00:00|2    |5      |27   |13     |2      |104              |96             |
+--------+-------------------+-----+-------+-----+-------+-------+-----------------+---------------+
only showing top 5 rows
```

Figure 7: Spark shell output showing Step 5 – Preprocessed dataset sample rows

3. SQL Queries

Q1: Monthly Crime Trend

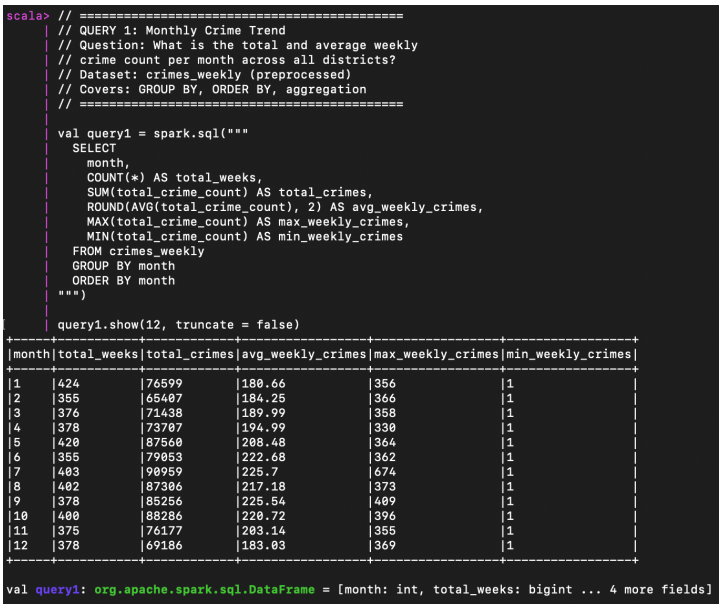
Table 3: Query metadata for Q1 – Monthly Crime Trend

Field	Details
Operation	GROUP BY + Aggregation
Question Answered	What is the total and average weekly crime count per month across all districts?
Dataset Used	crimes_weekly (preprocessed dataset – 4,644 rows)
SQL Features	GROUP BY, ORDER BY, SUM, AVG, MAX, MIN, COUNT

This query aggregates all weekly crime records by month to reveal seasonal patterns in crime activity. By computing total crimes, average weekly crimes, maximum, and minimum per month, it provides a comprehensive view of how crime volume fluctuates across the calendar year utilizing standard Spark SQL aggregate functions [2]. Understanding seasonal trends is essential for the forecasting model because it directly validates the month feature engineered in the preprocessing Phase .

Code:

```
val query1 = spark.sql("""
  SELECT
    month,
    COUNT(*) AS total_weeks,
    SUM(total_crime_count) AS total_crimes,
    ROUND(AVG(total_crime_count), 2) AS avg_weekly_crimes,
    MAX(total_crime_count) AS max_weekly_crimes,
    MIN(total_crime_count) AS min_weekly_crimes
  FROM crimes_weekly
  GROUP BY month
  ORDER BY month
""")
query1.show(12, truncate = false)
```



```
scala> // ========================================
     | // QUERY 1: Monthly Crime Trend
     | // Question: What is the total and average weekly
     | // crime count per month across all districts?
     | // Dataset: crimes_weekly (preprocessed)
     | // Covers: GROUP BY, ORDER BY, aggregation
     | // ========================================
     |
     | val query1 = spark.sql("""
     |   SELECT
     |     month,
     |     COUNT(*) AS total_weeks,
     |     SUM(total_crime_count) AS total_crimes,
     |     ROUND(AVG(total_crime_count), 2) AS avg_weekly_crimes,
     |     MAX(total_crime_count) AS max_weekly_crimes,
     |     MIN(total_crime_count) AS min_weekly_crimes
     |   FROM crimes_weekly
     |   GROUP BY month
     |   ORDER BY month
     | """)
     |
     | query1.show(12, truncate = false)
+-----+-----------+------------+----------------+----------------+----------------+
|month|total_weeks|total_crimes|avg_weekly_crimes|max_weekly_crimes|min_weekly_crimes|
+-----+-----------+------------+----------------+----------------+----------------+
|1    |424        |76599       |180.66          |356             |1               |
|2    |355        |65407       |184.25          |366             |1               |
|3    |376        |71438       |189.99          |358             |1               |
|4    |378        |73707       |194.99          |330             |1               |
|5    |420        |87560       |208.48          |364             |1               |
|6    |355        |79053       |222.68          |362             |1               |
|7    |403        |90959       |225.7           |674             |1               |
|8    |402        |87306       |217.18          |373             |1               |
|9    |378        |85256       |225.54          |409             |1               |
|10   |400        |88286       |220.72          |396             |1               |
|11   |375        |76177       |203.14          |355             |1               |
|12   |378        |69186       |183.03          |369             |1               |
+-----+-----------+------------+----------------+----------------+----------------+

val query1: org.apache.spark.sql.DataFrame = [month: int, total_weeks: bigint ... 4 more fields]
```

Figure 8: Spark shell output showing Q1 – Monthly Crime Trend

Sample Output:

month	total_weeks	total_crimes	avg_weekly_crimes	max_weekly_crimes	min_weekly_crimes
1	424	76,599	180.66	356	1
2	355	65,407	184.25	366	1
3	376	71,438	189.99	358	1
4	378	73,707	194.99	330	1
5	420	87,560	208.48	364	1
6	355	79,053	222.68	362	1
7	403	90,959	225.70	674	1
8	402	87,306	217.18	373	1
9	378	85,256	225.54	409	1
10	400	88,286	220.72	396	1
11	375	76,177	203.14	355	1
12	378	69,186	183.03	369	1

The results reveal a clear seasonal pattern: crime rises from January through July, peaks in July (90,959 total crimes, average 225.70 per week), then gradually declines toward December (69,186 total crimes, average 183.03 per week). February records the lowest average weekly crimes (184.25), consistent with reduced outdoor activity during winter. This seasonal pattern directly validates the inclusion of the month feature in the Phase 2 preprocessing pipeline, confirming that the model needs seasonal indicators to capture cyclical fluctuations in crime volume.

Q2: Top 10 Highest Crime Weeks

Table 4: Query metadata for Q2 – Top 10 Highest Crime Weeks

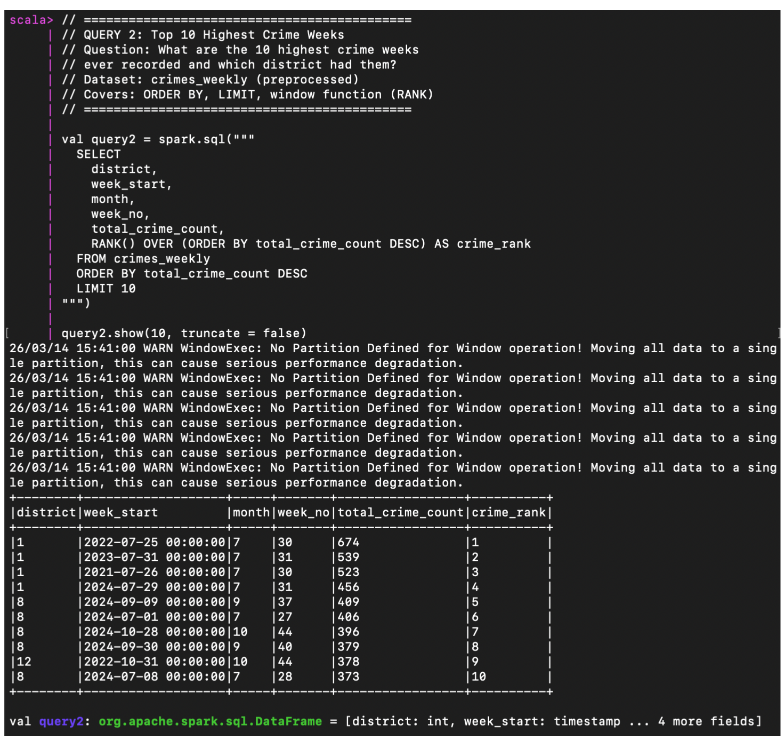
Field	Details
Operation	Window Function (RANK) + ORDER BY + LIMIT
Question Answered	What are the 10 highest crime weeks ever recorded and which district had them?
Dataset Used	crimes_weekly (preprocessed dataset – 4,644 rows)
SQL Features	RANK() OVER, ORDER BY, LIMIT

This query uses a RANK window function [1] to assign a global rank to every district-week combination [2] based on total crime count, then retrieves the top 10 highest-crime weeks across the entire 2021–2024 period. Identifying the most extreme crime weeks helps validate the need for the lag feature (prev_week_total) by demonstrating that certain periods experience dramatic spikes that the model must be able to detect from recent history.

Note: The WARN message about no PARTITION BY in the window function is expected behavior. Since the goal is a global ranking across all districts and weeks, no partition is intentionally defined.

Code:

```
val query2 = spark.sql("""
  SELECT
    district,
    week_start,
    month,
    week_no,
    total_crime_count,
    RANK() OVER (ORDER BY total_crime_count DESC) AS crime_rank
  FROM crimes_weekly
  ORDER BY total_crime_count DESC
  LIMIT 10
""")
query2.show(10, truncate = false)
```



```
scala> // ==========================================
     | // QUERY 2: Top 10 Highest Crime Weeks
     | // Question: What are the 10 highest crime weeks
     | // ever recorded and which district had them?
     | // Dataset: crimes_weekly (preprocessed)
     | // Covers: ORDER BY, LIMIT, window function (RANK)
     | // ==========================================
     |
     | val query2 = spark.sql("""
     |   SELECT
     |     district,
     |     week_start,
     |     month,
     |     week_no,
     |     total_crime_count,
     |     RANK() OVER (ORDER BY total_crime_count DESC) AS crime_rank
     |   FROM crimes_weekly
     |   ORDER BY total_crime_count DESC
     |   LIMIT 10
     | """)
     |
     | query2.show(10, truncate = false)
26/03/14 15:41:00 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a sing
le partition, this can cause serious performance degradation.
26/03/14 15:41:00 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a sing
le partition, this can cause serious performance degradation.
26/03/14 15:41:00 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a sing
le partition, this can cause serious performance degradation.
26/03/14 15:41:00 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a sing
le partition, this can cause serious performance degradation.
26/03/14 15:41:00 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a sing
le partition, this can cause serious performance degradation.
+--------+-------------------+-----+-------+-----------------+----------+
|district|week_start         |month|week_no|total_crime_count|crime_rank|
+--------+-------------------+-----+-------+-----------------+----------+
|1       |2022-07-25 00:00:00|7    |30     |674              |1         |
|1       |2023-07-31 00:00:00|7    |31     |539              |2         |
|1       |2021-07-26 00:00:00|7    |30     |523              |3         |
|1       |2024-07-29 00:00:00|7    |31     |456              |4         |
|8       |2024-09-09 00:00:00|9    |37     |409              |5         |
|8       |2024-07-01 00:00:00|7    |27     |406              |6         |
|8       |2024-10-28 00:00:00|10   |44     |396              |7         |
|8       |2024-09-30 00:00:00|9    |40     |379              |8         |
|12      |2022-10-31 00:00:00|10   |44     |378              |9         |
|8       |2024-07-08 00:00:00|7    |28     |373              |10        |
+--------+-------------------+-----+-------+-----------------+----------+

val query2: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 4 more fields]
```

Figure 9: Spark shell output showing Q2 – Top 10 Highest Crime Weeks

Sample Output:

district	week_start	month	week_no	total_crime_count	crime_rank
1	2022-07-25	7	30	674	1
1	2023-07-31	7	31	539	2
1	2021-07-26	7	30	523	3
1	2024-07-29	7	31	456	4
8	2024-09-09	9	37	409	5
8	2024-07-01	7	27	406	6
8	2024-10-28	10	44	396	7
8	2024-09-30	9	40	379	8
12	2022-10-31	10	44	378	9
8	2024-07-08	7	28	373	10

District 1 holds the four highest crime weeks on record, all occurring in July across consecutive years (2021–2024). The single highest week recorded 674 crimes during the week of July 25, 2022, nearly three times the city-wide weekly average of approximately 205(known from the Query 8). District 8 dominates positions 5 through 10. Notably, 8 of the top 10 weeks fall in July, September, or October, reinforcing the seasonal pattern observed in Query 1 and confirming that the week_no and month features are essential for the forecasting model to detect high-crime periods.

Q3: Districts Above City Average Arrest Rate

Table 5: Query metadata for Q3 – Arrest Rate Above City Average

Field	Details
Operation	CTE + Subquery + GROUP BY
Question Answered	Which districts have an arrest rate above the city-wide average?
Dataset Used	crimes_cleaned (cleaned dataset – 952,563 records)
SQL Features	WITH (CTE), GROUP BY, CASE WHEN, AVG, subquery

This query uses two Common Table Expressions (CTEs). The first CTE (district_arrests) computes total crimes, total arrests, and arrest rate for each district. The second CTE (city_average) computes the city-wide average arrest rate across all districts. The final SELECT then joins both CTEs to classify each district as above or below the city average, providing a comparative view of enforcement effectiveness across Chicago.

Code:

```
val query3 = spark.sql("""
  WITH district_arrests AS (
    SELECT district,
      COUNT(*) AS total_crimes,
      SUM(CASE WHEN arrest = true THEN 1 ELSE 0 END) AS total_arrests,
      ROUND(SUM(CASE WHEN arrest = true THEN 1 ELSE 0 END)
          * 100.0 / COUNT(*), 2) AS arrest_rate
    FROM crimes_cleaned GROUP BY district
  ),
  city_average AS (
    SELECT ROUND(AVG(arrest_rate), 2) AS avg_arrest_rate
    FROM district_arrests
  )
  SELECT d.district, d.total_crimes, d.total_arrests,
    d.arrest_rate, c.avg_arrest_rate,
    CASE WHEN d.arrest_rate > c.avg_arrest_rate
        THEN 'ABOVE AVERAGE' ELSE 'BELOW AVERAGE' END AS performance
  FROM district_arrests d, city_average c
  ORDER BY d.arrest_rate DESC
""")
query3.show(25, truncate = false)
```

```
scala> // ========================================
     | // QUERY 3: Districts Above City Average Arrest Rate
     | // Question: Which districts have an arrest rate
     | // above the city-wide average?
     | // Dataset: crimes_cleaned
     | // Covers: CTE, subquery, GROUP BY
     | // ========================================
     |
     | val query3 = spark.sql("""
     |   WITH district_arrests AS (
     |     SELECT
     |       district,
     |       COUNT(*) AS total_crimes,
     |       SUM(CASE WHEN arrest = true THEN 1 ELSE 0 END) AS total_arrests,
     |       ROUND(SUM(CASE WHEN arrest = true THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS arrest_rate
     |     FROM crimes_cleaned
     |     GROUP BY district
     |   ),
     |   city_average AS (
     |     SELECT ROUND(AVG(arrest_rate), 2) AS avg_arrest_rate
     |     FROM district_arrests
     |   )
     |   SELECT
     |     d.district,
     |     d.total_crimes,
     |     d.total_arrests,
     |     d.arrest_rate,
     |     c.avg_arrest_rate,
     |     CASE WHEN d.arrest_rate > c.avg_arrest_rate THEN 'ABOVE AVERAGE'
     |          ELSE 'BELOW AVERAGE' END AS performance
     |   FROM district_arrests d, city_average c
     |   ORDER BY d.arrest_rate DESC
     | """)
     |
     | query3.show(25, truncate = false)
+--------+-----------+-------------+-----------+---------------+-------------+
|district|total_crimes|total_arrests|arrest_rate|avg_arrest_rate|performance  |
+--------+-----------+-------------+-----------+---------------+-------------+
|31      |57         |17           |29.82      |13.31          |ABOVE AVERAGE|
|11      |53691      |13093        |24.39      |13.31          |ABOVE AVERAGE|
|10      |40190      |7068         |17.59      |13.31          |ABOVE AVERAGE|
|1       |50305      |8036         |15.97      |13.31          |ABOVE AVERAGE|
|15      |32922      |5208         |15.82      |13.31          |ABOVE AVERAGE|
|7       |42026      |6262         |14.90      |13.31          |ABOVE AVERAGE|
|5       |39681      |5437         |13.70      |13.31          |ABOVE AVERAGE|
|25      |49408      |6591         |13.34      |13.31          |ABOVE AVERAGE|
|18      |48209      |6362         |13.20      |13.31          |BELOW AVERAGE|
|9       |42006      |5239         |12.47      |13.31          |BELOW AVERAGE|
|6       |58262      |7097         |12.18      |13.31          |BELOW AVERAGE|
|16      |34805      |4156         |11.94      |13.31          |BELOW AVERAGE|
|22      |30435      |3624         |11.91      |13.31          |BELOW AVERAGE|
|20      |20035      |2185         |10.91      |13.31          |BELOW AVERAGE|
|24      |32921      |3579         |10.87      |13.31          |BELOW AVERAGE|
|14      |33555      |3606         |10.75      |13.31          |BELOW AVERAGE|
|8       |61143      |6499         |10.63      |13.31          |BELOW AVERAGE|
|3       |48304      |5127         |10.61      |13.31          |BELOW AVERAGE|
|17      |28082      |2896         |10.31      |13.31          |BELOW AVERAGE|
|4       |54444      |5113         |9.39       |13.31          |BELOW AVERAGE|
|2       |47391      |4093         |8.64       |13.31          |BELOW AVERAGE|
|19      |48423      |4165         |8.60       |13.31          |BELOW AVERAGE|
|12      |56268      |4561         |8.11       |13.31          |BELOW AVERAGE|
+--------+-----------+-------------+-----------+---------------+-------------+

val query3: org.apache.spark.sql.DataFrame = [district: int, total_crimes: bigint ... 4 more fields]
```

Figure 10: Spark shell output showing Q3 – Arrest Rate Above City Average

Sample Output (selected rows):

district	total_crimes	total_arrests	arrest_rate	avg_arrest_rate	performance
31	57	17	29.82%	13.31%	ABOVE AVERAGE
11	53,691	13,093	24.39%	13.31%	ABOVE AVERAGE
10	40,190	7,068	17.59%	13.31%	ABOVE AVERAGE
1	50,305	8,036	15.97%	13.31%	ABOVE AVERAGE
8	61,143	6,499	10.63%	13.31%	BELOW AVERAGE
4	54,444	5,113	9.39%	13.31%	BELOW AVERAGE
2	47,391	4,093	8.64%	13.31%	BELOW AVERAGE
12	56,268	4,561	8.11%	13.31%	BELOW AVERAGE

The city-wide average arrest rate is 13.31%. Only 8 of 23 districts exceed this threshold. District 11 stands out as the highest arrest rate among high-crime districts (24.39%), suggesting more proactive or effective enforcement despite its 53,691 total crimes. District 8, the highest crime district overall, has an arrest rate of only 10.63%, well below the city average, which powerfully reinforces the project's core motivation: reactive policing is structurally limited in high-crime areas, making proactive crime forecasting most valuable precisely where enforcement outcomes are weakest. District 12 has the lowest arrest rate (8.11%) despite being the third highest crime district.

Q4: Statistical Summary of Key Crime Types per District

Table 6: Query metadata for Q4 – Statistical Summary of Key Crime Types

Field	Details
Operation	Statistical Aggregation
Question Answered	What is the average, standard deviation, and variance of weekly THEFT, BATTERY, and ROBBERY counts per district?
Dataset Used	crimes_weekly (preprocessed dataset – 4,644 rows)
SQL Features	AVG, STDDEV, VARIANCE, GROUP BY, ORDER BY, ROUND, LIMIT

This query computes descriptive statistics for three dominant crime types: THEFT, BATTERY, and ROBBERY, as well as the overall total_crime_count per district. By measuring both the average and the spread (standard deviation, variance) of weekly counts, it reveals not only which districts have the highest crime volumes but also which are the most unpredictable from week to week. High variance districts are inherently more difficult to forecast and represent the greatest challenge for our regression model.

Code:

```
val query4 = spark.sql("""
  SELECT district,
    ROUND(AVG(THEFT), 2) AS avg_theft,
    ROUND(STDDEV(THEFT), 2) AS stddev_theft,
    ROUND(AVG(BATTERY), 2) AS avg_battery,
    ROUND(STDDEV(BATTERY), 2) AS stddev_battery,
    ROUND(AVG(ROBBERY), 2) AS avg_robbery,
    ROUND(STDDEV(ROBBERY), 2) AS stddev_robbery,
    ROUND(AVG(total_crime_count), 2) AS avg_total,
    ROUND(VARIANCE(total_crime_count), 2) AS variance_total
  FROM crimes_weekly
  GROUP BY district
  ORDER BY avg_total DESC
  LIMIT 10
""")
query4.show(10, truncate = false)
```

```
scala> // ==========================================
     | // QUERY 4: Statistical Summary of Key Crime Types
     | // Question: What is the average, variance and
     | // stddev of weekly THEFT, BATTERY and ROBBERY
     | // counts per district?
     | // Dataset: crimes_weekly (preprocessed)
     | // Covers: Statistical summaries (avg, variance, stddev)
     | // ==========================================
     |
     | val query4 = spark.sql("""
     |   SELECT
     |     district,
     |     ROUND(AVG(THEFT), 2)                  AS avg_theft,
     |     ROUND(STDDEV(THEFT), 2)               AS stddev_theft,
     |     ROUND(AVG(BATTERY), 2)                AS avg_battery,
     |     ROUND(STDDEV(BATTERY), 2)             AS stddev_battery,
     |     ROUND(AVG(ROBBERY), 2)                AS avg_robbery,
     |     ROUND(STDDEV(ROBBERY), 2)             AS stddev_robbery,
     |     ROUND(AVG(total_crime_count), 2)      AS avg_total,
     |     ROUND(VARIANCE(total_crime_count), 2) AS variance_total
     |   FROM crimes_weekly
     |   GROUP BY district
     |   ORDER BY avg_total DESC
     |   LIMIT 10
     | """)
     |
     | query4.show(10, truncate = false)
+--------+---------+------------+-----------+--------------+-----------+--------------+---------+--------------+
|district|avg_theft|stddev_theft|avg_battery|stddev_battery|avg_robbery|stddev_robbery|avg_total|variance_total|
+--------+---------+------------+-----------+--------------+-----------+--------------+---------+--------------+
|8       |53.8     |13.72       |50.95      |9.39          |9.84       |4.68          |292.03   |2555.99       |
|6       |42.24    |8.21        |56.72      |9.72          |11.44      |4.72          |278.2    |1305.87       |
|12      |79.58    |19.79       |38.33      |10.15         |13.43      |7.48          |268.86   |3389.82       |
|4       |40.41    |10.67       |51.47      |9.52          |9.48       |4.6           |259.98   |1530.0        |
|11      |29.43    |7.06        |53.57      |11.24         |13.84      |5.48          |256.34   |1363.65       |
|1       |86.37    |43.82       |34.02      |8.47          |8.0        |3.65          |240.47   |5088.88       |
|25      |45.88    |11.81       |41.98      |8.59          |10.39      |7.45          |235.97   |1440.04       |
|19      |86.12    |28.42       |30.9       |8.72          |7.24       |4.07          |231.45   |3009.07       |
|3       |36.29    |9.15        |48.56      |9.12          |8.35       |3.09          |230.68   |1279.59       |
|18      |89.14    |24.64       |30.92      |8.35          |7.9        |4.24          |230.33   |2281.78       |
+--------+---------+------------+-----------+--------------+-----------+--------------+---------+--------------+

val query4: org.apache.spark.sql.DataFrame = [district: int, avg_theft: double ... 7 more fields]
```

Figure 11: Spark shell output showing Q4 – Statistical Summary of Key Crime Types

Sample Output (top 10 districts by average weekly total):

district	avg_theft	stddev_theft	avg_battery	stddev_battery	avg_robbery	stddev_robbery	avg_total	variance_total
8	53.80	13.72	50.95	9.39	9.84	4.68	292.03	2,555.99
6	42.24	8.21	56.72	9.72	11.44	4.72	278.20	1,305.87
12	79.58	19.79	38.33	10.15	13.43	7.48	268.86	3,389.82
4	40.41	10.67	51.47	9.52	9.48	4.60	259.98	1,530.00
11	29.43	7.06	53.57	11.24	13.84	5.48	256.34	1,363.65
1	86.37	43.82	34.02	8.47	8.00	3.65	240.47	5,088.88
25	45.88	11.81	41.98	8.59	10.39	7.45	235.97	1,440.04
19	86.12	28.42	30.90	8.72	7.24	4.07	231.45	3,009.07
3	36.29	9.15	48.56	9.12	8.35	3.09	230.68	1,279.59
18	89.14	24.64	30.92	8.35	7.90	4.24	230.33	2,281.78

District 8 leads in average weekly total crimes (292.03) and District 1 has the highest variance (5,088.88), meaning District 1 is the most unpredictable despite not being the highest-volume district. Each district exhibits a distinct crime-type profile: Districts 12, 18, and 19 are THEFT-dominated (averages above 79 per week), while Districts 6, 4, and 11 are BATTERY-dominated. District 11 leads in average weekly ROBBERY (13.84). This behavioral diversity across districts confirms that the preprocessing Phase decision to preserve individual crime-type pivot columns as separate features was correct, a single aggregated total count would obscure these meaningful differences.

Q5: Year-over-Year Crime Change per District

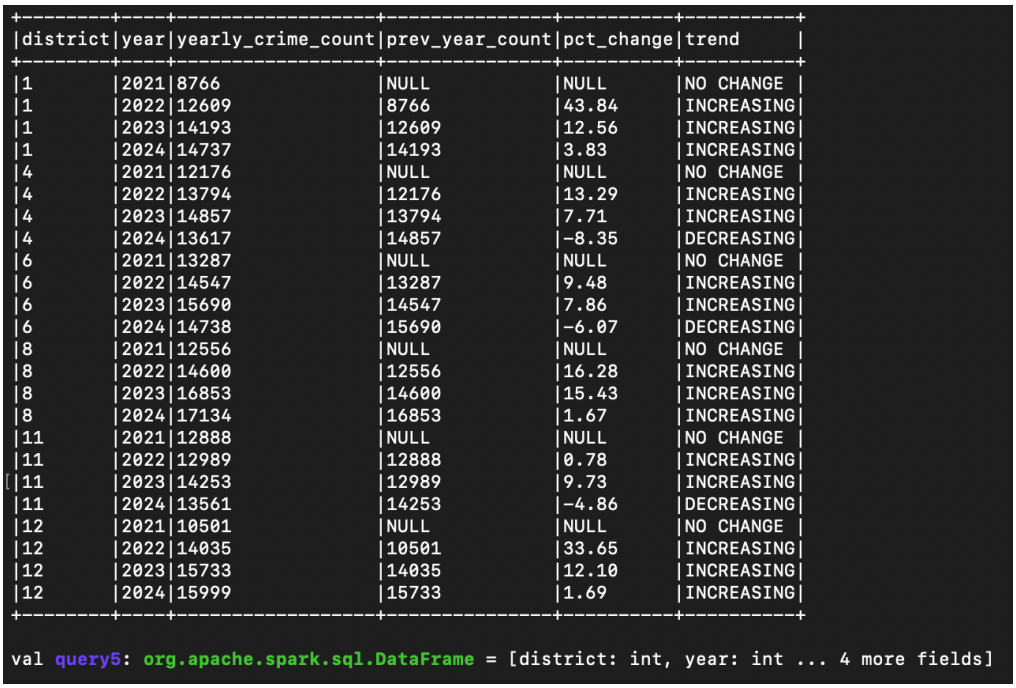
Table 7: Query metadata for Q5 – Year-over-Year Crime Change

Field	Details
Operation	Window Function (LAG) + CTE
Question Answered	How did total yearly crime change per district between 2021 and 2024?
Dataset Used	crimes_cleaned (cleaned dataset – 952,563 records)
SQL Features	WITH (CTE), LAG() OVER (PARTITION BY), GROUP BY, CASE WHEN, ROUND

This query uses two CTEs and a LAG window function [1] partitioned by district to compute the year-over-year percentage change [2] in total crime count for six key districts. The LAG function retrieves the previous year's crime count for each district, enabling calculation of the percentage change and classification of each year's trend as INCREASING or DECREASING. This query extends the year-over-year analysis from the RDD operations Phase by quantifying the rate of change rather than just the direction.

Code:

```
val query5 = spark.sql("""
  WITH yearly_totals AS (
    SELECT district, year, COUNT(*) AS yearly_crime_count
    FROM crimes_cleaned GROUP BY district, year
  ),
  yearly_with_change AS (
    SELECT district, year, yearly_crime_count,
      LAG(yearly_crime_count) OVER (PARTITION BY district ORDER BY year) AS
prev_year_count,
      ROUND((yearly_crime_count - LAG(yearly_crime_count)
        OVER (PARTITION BY district ORDER BY year)) * 100.0
        / LAG(yearly_crime_count) OVER (PARTITION BY district ORDER BY year), 2)
AS pct_change
    FROM yearly_totals
  )
  SELECT district, year, yearly_crime_count, prev_year_count, pct_change,
    CASE WHEN pct_change > 0 THEN 'INCREASING'
         WHEN pct_change < 0 THEN 'DECREASING'
         ELSE 'NO CHANGE' END AS trend
  FROM yearly_with_change
  WHERE district IN (1, 4, 6, 8, 11, 12)
  ORDER BY district, year
""")
query5.show(30, truncate = false)
```



```
+--------+----+-----------------+---------------+----------+----------+
|district|year|yearly_crime_count|prev_year_count|pct_change|trend     |
+--------+----+-----------------+---------------+----------+----------+
|1       |2021|8766             |NULL           |NULL      |NO CHANGE |
|1       |2022|12609            |8766           |43.84     |INCREASING|
|1       |2023|14193            |12609          |12.56     |INCREASING|
|1       |2024|14737            |14193          |3.83      |INCREASING|
|4       |2021|12176            |NULL           |NULL      |NO CHANGE |
|4       |2022|13794            |12176          |13.29     |INCREASING|
|4       |2023|14857            |13794          |7.71      |INCREASING|
|4       |2024|13617            |14857          |-8.35     |DECREASING|
|6       |2021|13287            |NULL           |NULL      |NO CHANGE |
|6       |2022|14547            |13287          |9.48      |INCREASING|
|6       |2023|15690            |14547          |7.86      |INCREASING|
|6       |2024|14738            |15690          |-6.07     |DECREASING|
|8       |2021|12556            |NULL           |NULL      |NO CHANGE |
|8       |2022|14600            |12556          |16.28     |INCREASING|
|8       |2023|16853            |14600          |15.43     |INCREASING|
|8       |2024|17134            |16853          |1.67      |INCREASING|
|11      |2021|12888            |NULL           |NULL      |NO CHANGE |
|11      |2022|12989            |12888          |0.78      |INCREASING|
|11      |2023|14253            |12989          |9.73      |INCREASING|
|11      |2024|13561            |14253          |-4.86     |DECREASING|
|12      |2021|10501            |NULL           |NULL      |NO CHANGE |
|12      |2022|14035            |10501          |33.65     |INCREASING|
|12      |2023|15733            |14035          |12.10     |INCREASING|
|12      |2024|15999            |15733          |1.69      |INCREASING|
+--------+----+-----------------+---------------+----------+----------+

val query5: org.apache.spark.sql.DataFrame = [district: int, year: int ... 4 more fields]
```

Figure 12: Spark shell output showing Q5 – Year-over-Year Crime Change

Sample Output:

district	year	yearly_crimes	prev_year	pct_change	trend
1	2021	8,766	NULL	NULL	NO CHANGE
1	2022	12,609	8,766	+43.84%	INCREASING
1	2023	14,193	12,609	+12.56%	INCREASING
1	2024	14,737	14,193	+3.83%	INCREASING
4	2024	13,617	14,857	-8.35%	DECREASING
6	2024	14,738	15,690	-6.07%	DECREASING
8	2022	14,600	12,556	+16.28%	INCREASING
8	2023	16,853	14,600	+15.43%	INCREASING
11	2024	13,561	14,253	-4.86%	DECREASING
12	2022	14,035	10,501	+33.65%	INCREASING

The results confirm that crime trends are non-linear and vary significantly across districts. District 1 experienced the most dramatic increase from 2021 to 2022 (+43.84%), while District 8 showed consistent growth across all four years. Districts 4, 6, and 11 peaked in 2023 and declined in 2024, demonstrating a non-monotonic pattern. These non-linear trajectories validate the preprocessing Phase decision to engineer the lag feature (prev_week_total) rather than relying on a simple linear trend. The model needs to capture recent momentum rather than assume a constant direction of change.

Q6: Peak Crime Hour Analysis

Table 8: Query metadata for Q6 – Peak Crime Hour Analysis

Field	Details
Operation	HOUR() Function + Window Percentage
Question Answered	What hours of the day have the highest crime counts?
Dataset Used	crimes_cleaned (cleaned dataset – 952,563 records)
SQL Features	HOUR(), GROUP BY, ORDER BY, SUM() OVER () window for percentage

This query extracts the hour component from the date timestamp using the HOUR() function [2] and aggregates crime counts by hour of day. A window function computes the percentage share of each hour relative to the total. This temporal analysis reveals intra-day crime distribution patterns, which could motivate additional feature engineering in the Machine Learning Phase such as an is_night or hour_of_day feature.

Code:

```
val query6 = spark.sql("""
  SELECT
    HOUR(date) AS hour_of_day,
    COUNT(*) AS total_crimes,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS percentage
  FROM crimes_cleaned
  GROUP BY HOUR(date)
  ORDER BY total_crimes DESC
""")
query6.show(24, truncate = false)
```

```
scala> // ========================================
     | // QUERY 6: Peak Crime Hour Analysis
     | // Question: What hours of the day have the
     | // highest crime counts?
     | // Dataset: crimes_cleaned
     | // Covers: GROUP BY, ORDER BY, EXTRACT function
     | // ========================================
     |
     | val query6 = spark.sql("""
     |   SELECT
     |     HOUR(date) AS hour_of_day,
     |     COUNT(*) AS total_crimes,
     |     ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS percentage
     |   FROM crimes_cleaned
     |   GROUP BY HOUR(date)
     |   ORDER BY total_crimes DESC
     | """)
     |
     | query6.show(24, truncate = false)
26/03/14 15:41:42 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
26/03/14 15:41:42 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
26/03/14 15:41:42 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
26/03/14 15:41:43 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
26/03/14 15:41:43 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
26/03/14 15:41:43 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
26/03/14 15:41:43 WARN WindowExec: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
+-----------+------------+----------+
|hour_of_day|total_crimes|percentage|
+-----------+------------+----------+
|0          |69810       |7.33      |
|12         |54288       |5.70      |
|17         |50990       |5.35      |
|15         |50904       |5.34      |
|18         |49974       |5.25      |
|16         |49878       |5.24      |
|19         |48989       |5.14      |
|20         |47871       |5.03      |
|14         |44705       |4.69      |
|21         |44574       |4.68      |
|13         |42729       |4.49      |
|22         |42701       |4.48      |
|11         |41351       |4.34      |
|10         |40587       |4.26      |
|9          |39907       |4.19      |
|23         |38809       |4.07      |
|8          |32170       |3.38      |
|1          |31934       |3.35      |
|2          |28446       |2.99      |
|3          |23801       |2.50      |
|7          |23697       |2.49      |
|4          |19581       |2.06      |
|6          |17911       |1.88      |
|5          |16956       |1.78      |
+-----------+------------+----------+

val query6: org.apache.spark.sql.DataFrame = [hour_of_day: int, total_crimes: bigint ... 1 more field]
```

Figure 13: Spark shell output showing Q6 – Peak Crime Hour Analysis

Sample Output (top 10 hours):

hour_of_day	total_crimes	percentage
0 (midnight)	69,810	7.33%
12 (noon)	54,288	5.70%
17 (5pm)	50,990	5.35%
15 (3pm)	50,904	5.34%
18 (6pm)	49,974	5.25%
16 (4pm)	49,878	5.24%
19 (7pm)	48,989	5.14%
20 (8pm)	47,871	5.03%
5 (5am)	16,956	1.78%
6 (6am)	17,911	1.88%

Midnight (hour 0) records the highest crime count (69,810, or 7.33% of all incidents), which is a known data characteristic where crimes without a precise recorded time are assigned a default midnight timestamp. Excluding this known artifact, the afternoon and evening hours (12:00–20:00) consistently show the highest genuine crime activity, each accounting for approximately 5% of all incidents. Early morning hours (5–6 AM) are the safest, recording below 2% each. This intra-day pattern suggests that an hour-of-day or time-of-day categorical feature could further improve model accuracy in the Machine Learning Phase.

Q7: Domestic vs Non-Domestic Crime per District

Table 9: Query metadata for Q7 – Domestic vs Non-Domestic Crime

Field	Details
Operation	GROUP BY + HAVING + Conditional Aggregation
Question Answered	What percentage of crimes are domestic-related per district, for districts above 15%?
Dataset Used	crimes_cleaned (cleaned dataset – 952,563 records)
SQL Features	GROUP BY, HAVING, CASE WHEN, ROUND, ORDER BY

This query uses conditional aggregation with CASE WHEN [1] to separately count domestic and non-domestic crimes per district, then computes the domestic crime percentage. The HAVING clause filters to only show districts where more than 15% of crimes are domestic-related. This reveals significant behavioral differences across districts that are not captured by total crime counts alone, further justifying the preprocessing Phase decision to treat each district as a unique category.

Code:

```
val query7 = spark.sql("""
  SELECT
    district,
    COUNT(*) AS total_crimes,
    SUM(CASE WHEN domestic = true THEN 1 ELSE 0 END) AS domestic_crimes,
    SUM(CASE WHEN domestic = false THEN 1 ELSE 0 END) AS
non_domestic_crimes,
    ROUND(SUM(CASE WHEN domestic = true THEN 1 ELSE 0 END)
        * 100.0 / COUNT(*), 2) AS domestic_pct
  FROM crimes_cleaned
  GROUP BY district
  HAVING domestic_pct > 15
  ORDER BY domestic_pct DESC
""")
query7.show(25, truncate = false)
```

```
scala> // ========================================
     | // QUERY 7: Domestic vs Non-Domestic Crime
     | // Question: What percentage of crimes are
     | // domestic-related per district?
     | // Dataset: crimes_cleaned
     | // Covers: GROUP BY, HAVING, conditional aggregation
     | // ========================================
     |
     | val query7 = spark.sql("""
     |   SELECT
     |     district,
     |     COUNT(*) AS total_crimes,
     |     SUM(CASE WHEN domestic = true THEN 1 ELSE 0 END) AS domestic_crimes,
     |     SUM(CASE WHEN domestic = false THEN 1 ELSE 0 END) AS non_domestic_crimes,
     |     ROUND(SUM(CASE WHEN domestic = true THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS domestic_pct
     |   FROM crimes_cleaned
     |   GROUP BY district
     |   HAVING domestic_pct > 15
     |   ORDER BY domestic_pct DESC
     | """)
     |
     | query7.show(25, truncate = false)
+--------+------------+---------------+-------------------+------------+
|district|total_crimes|domestic_crimes|non_domestic_crimes|domestic_pct|
+--------+------------+---------------+-------------------+------------+
|15      |32922       |9689           |23233              |29.43       |
|7       |42026       |12212          |29814              |29.06       |
|5       |39681       |11118          |28563              |28.02       |
|6       |58262       |15784          |42478              |27.09       |
|3       |48304       |13007          |35297              |26.93       |
|4       |54444       |14550          |39894              |26.72       |
|10      |40190       |9829           |30361              |24.46       |
|11      |53691       |12514          |41177              |23.31       |
|22      |30435       |6789           |23646              |22.31       |
|25      |49408       |10537          |38871              |21.33       |
|2       |47391       |9940           |37451              |20.97       |
|8       |61143       |12640          |48503              |20.67       |
|9       |42006       |8506           |33500              |20.25       |
|16      |34805       |5682           |29123              |16.33       |
|17      |28082       |4425           |23657              |15.76       |
|24      |32921       |5143           |27778              |15.62       |
+--------+------------+---------------+-------------------+------------+

val query7: org.apache.spark.sql.DataFrame = [district: int, total_crimes: bigint ... 3 more fields]
```

Figure 14: Spark shell output showing Q7 – Domestic vs Non-Domestic Crime

Sample Output:

district	total_crimes	domestic_crimes	non_domestic_crimes	domestic_pct
15	32,922	9,689	23,233	29.43%
7	42,026	12,212	29,814	29.06%
5	39,681	11,118	28,563	28.02%
6	58,262	15,784	42,478	27.09%
3	48,304	13,007	35,297	26.93%
4	54,444	14,550	39,894	26.72%
8	61,143	12,640	48,503	20.67%
16	34,805	5,682	29,123	16.33%
24	32,921	5,143	27,778	15.62%

16 of 23 districts have domestic crime rates above 15%, with Districts 15, 7, and 5 exceeding 28%. The District 8 has overall highest crime district with a 20.67% domestic rate, meaning approximately 1 in 5 crimes there is domestic-related. The 7 districts falling below the 15% threshold tend to be commercial or downtown areas where domestic incidents are naturally less common. This wide variation in domestic crime composition across districts further confirms that each district has a distinct behavioral profile, supporting the preprocessing Phase decision to preserve individual crime-type feature columns and treat districts as independent categorical entities rather than interchangeable numeric values.

Q8: Districts with Weekly Crime Consistently Above City Average

Table 10: Query metadata for Q8 – Districts Above City Average

Field	Details
Operation	HAVING + Subquery
Question Answered	Which districts have an average weekly crime count above the overall city average?
Dataset Used	crimes_weekly (preprocessed dataset – 4,644 rows)
SQL Features	GROUP BY, HAVING, subquery in SELECT and HAVING, AVG, MAX, MIN, COUNT

This query uses a subquery in both the HAVING clause and the SELECT clause to identify districts whose average weekly crime count exceeds the overall city-wide weekly average. The subquery computes the city average inline, allowing direct comparison without a separate CTE. This identifies the consistently high-crime districts that require the most accurate forecasts, as errors in predicting these districts have the largest operational impact on resource allocation.

Code:

```
val query8 = spark.sql("""
  SELECT
    district,
    ROUND(AVG(total_crime_count), 2) AS avg_weekly_crimes,
    MAX(total_crime_count) AS max_weekly_crimes,
    MIN(total_crime_count) AS min_weekly_crimes,
    COUNT(*) AS total_weeks,
    ROUND((SELECT AVG(total_crime_count) FROM crimes_weekly), 2) AS city_avg
  FROM crimes_weekly
  GROUP BY district
  HAVING AVG(total_crime_count) > (
    SELECT AVG(total_crime_count) FROM crimes_weekly
  )
  ORDER BY avg_weekly_crimes DESC
""")
query8.show(25, truncate = false)
```

```
scala> // ========================================
     | // QUERY 8: Districts with Consistently High
     | // Weekly Crime Above City Average
     | // Question: Which districts have an average
     | // weekly crime count above the overall average?
     | // Dataset: crimes_weekly (preprocessed)
     | // Covers: HAVING, subquery, GROUP BY
     | // ========================================
     |
     | val query8 = spark.sql("""
     |   SELECT
     |     district,
     |     ROUND(AVG(total_crime_count), 2) AS avg_weekly_crimes,
     |     MAX(total_crime_count) AS max_weekly_crimes,
     |     MIN(total_crime_count) AS min_weekly_crimes,
     |     COUNT(*) AS total_weeks,
     |     ROUND((SELECT AVG(total_crime_count) FROM crimes_weekly), 2) AS city_avg
     |   FROM crimes_weekly
     |   GROUP BY district
     |   HAVING AVG(total_crime_count) > (
     |     SELECT AVG(total_crime_count) FROM crimes_weekly
     |   )
     |   ORDER BY avg_weekly_crimes DESC
     | """)
     |
     | query8.show(25, truncate = false)
+--------+----------------+----------------+----------------+-----------+--------+
|district|avg_weekly_crimes|max_weekly_crimes|min_weekly_crimes|total_weeks|city_avg|
+--------+----------------+----------------+----------------+-----------+--------+
|8       |292.03          |409             |76              |209        |204.77  |
|6       |278.2           |364             |71              |209        |204.77  |
|12      |268.86          |378             |88              |209        |204.77  |
|4       |259.98          |369             |61              |209        |204.77  |
|11      |256.34          |356             |72              |209        |204.77  |
|1       |240.47          |674             |63              |209        |204.77  |
|25      |235.97          |320             |73              |209        |204.77  |
|19      |231.45          |362             |70              |209        |204.77  |
|3       |230.68          |305             |68              |209        |204.77  |
|18      |230.33          |321             |65              |209        |204.77  |
|2       |226.43          |333             |75              |209        |204.77  |
+--------+----------------+----------------+----------------+-----------+--------+

val query8: org.apache.spark.sql.DataFrame = [district: int, avg_weekly_crimes: double ... 4 more field
s]
```

Figure 15: Spark shell output showing Q8 – Districts Above City Average

Sample Output:

district	avg_weekly_crimes	max_weekly_crimes	min_weekly_crimes	total_weeks	city_avg
8	292.03	409	76	209	204.77
6	278.20	364	71	209	204.77
12	268.86	378	88	209	204.77
4	259.98	369	61	209	204.77
11	256.34	356	72	209	204.77
1	240.47	674	63	209	204.77
25	235.97	320	73	209	204.77
19	231.45	362	70	209	204.77
3	230.68	305	68	209	204.77
18	230.33	321	65	209	204.77
2	226.43	333	75	209	204.77

The city-wide average weekly crime count is 204.77. Eleven of the 23 districts consistently exceed this average, with District 8 leading at 292.03 average weekly crimes — approximately 43% above the city average. All 11 above-average districts have exactly 209 weekly records, confirming complete data coverage for the full 4-year study period. District 1 shows the widest range (minimum 63, maximum 674), indicating the highest week-to-week volatility among the above-average districts. These 11 districts represent the highest forecasting priority and are where accurate weekly predictions will have the greatest impact on patrol deployment and resource planning.

4. Analysis Summary and Key Insights

4.1 Seasonal Crime Patterns Validate Temporal Features

Query 1 confirms a strong seasonal pattern: crime peaks in July (average 225.70 crimes per week) and reaches its lowest point in January and February (averages around 180–184). This directly validates the month and week_no features engineered in the preprocessing Phase, confirming that the model needs seasonal indicators to capture the predictable annual fluctuations in crime volume.

4.2 High-Crime Weeks Are Concentrated in Summer Months

Query 2 reveals that the 10 highest crime weeks on record are dominated by Districts 1 and 8, with 8 of the top 10 weeks occurring in July, September, or October. The single highest week (District 1, July 2022) recorded 674 crimes, more than three times the city average. This extreme spike behavior justifies the lag feature (prev_week_total), which allows the model to detect recent momentum before such peaks occur.

4.3 Low Arrest Rates Motivate Proactive Forecasting

Query 3 shows that the city-wide average arrest rate is only 13.31%, and 15 of 23 operational districts fall below this threshold. District 8 which is the highest crime district, has an arrest rate of just 10.63%, confirming that reactive policing is structurally limited in the areas that need intervention most. This finding powerfully motivates the project's core objective of proactive crime forecasting.

4.4 Each District Has a Unique Crime-Type Behavioral Profile

Query 4 reveals that districts have distinctly different dominant crime types. Districts 12, 18, and 19 are THEFT-dominated, Districts 6, 4, and 11 are BATTERY-dominated, and District 11 leads in ROBBERY. District 1 has the highest variance (5,088.88), making it the most unpredictable district to forecast. These behavioral differences confirm that the preprocessing Phase pivoted aggregation strategy which is preserving individual crime-type columns as separate features was the correct design decision.

4.5 Non-Linear Year-over-Year Trends Require Lag Features

Query 5 quantifies the year-over-year percentage change per district and confirms non-linear patterns across all six analyzed districts. District 1 grew by 43.84% from 2021 to 2022, while Districts 4, 6, and 11 peaked in 2023 and declined in 2024. These non-monotonic trends validate the preprocessing Phase lag feature design, which captures recent momentum rather than assuming a constant trend direction.

4.6 Intra-Day Patterns Reveal Peak Crime Hours

Query 6 identifies afternoon and evening hours (12:00–20:00) as consistently high-crime periods, each accounting for approximately 5% of all incidents. The midnight spike (7.33%) is a known data artifact from default timestamp assignment. Early morning hours (5–6 AM) are the safest. This finding suggests that a time-of-day feature could further improve model accuracy in the ML Phase .

4.7 Domestic Crime Composition Varies Significantly Across Districts

Query 7 shows that 16 of 23 districts have domestic crime rates above 15%, with Districts 15, 7, and 5 exceeding 28%. The wide variation in domestic crime composition across districts provides additional justification for treating each district as a unique behavioral category, consistent with the preprocessing Phase district_indexed encoding approach.

4.8 Eleven Districts Consistently Exceed the City Weekly Average

Query 8 identifies 11 districts whose average weekly crime counts exceed the city-wide average of 204.77. These districts represent the highest forecasting priority, as accurate predictions there have the greatest operational impact on patrol scheduling and resource deployment. District 8 leads at 292.03 average weekly crimes, approximately 43% above the city average.

5. Reference

[1] J. S. Damji, B. Wenig, T. Das, and D. Lee, *Learning Spark: Lightning-Fast Data Analytics*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020. [Online]. Available:

https://vdoc.pub/download/learning-spark-lightning-fast-data-analytics-61bd73j55np0.

[2] Apache Software Foundation, "Spark SQL, Built-in Functions," Apache Spark Documentation, 2026. [Online].
Available: https://spark.apache.org/docs/latest/api/sql/index.html. [Accessed: 15-Mar-2026].